

온체인 이벤트 수신 설계 즉시 처리의 위험과 비동기 분리

202 Accepted 패턴 · Queue 분리 · Promise.allSettled · rawBody Buffer

강의 25분 + 실습 30분

즉시 처리 위험 분석

WebhookServer 완전 해부

실습 4단계

5-Layer 아키텍처 리마인드

ARCHITECTURE REMINDER — 신뢰 경계로 시스템을 나눈다 · GATEWAY LAYER SEPARATES INTERNAL LAYERS

내부망

레이어 1 — 사용자

Kyobo App

↓ 사용자 이벤트 / API 응답

레이어 2 — 게이트웨이

Webhook
Receiver

Redis Event
Pipeline

IVASP
Adapter

IBlockchain
Adapter

← 발행 요청 / Webhook 이벤트 →

↑ 발행 요청 / 원장 업데이트

레이어 3 — 코어 서비스

비즈니스 로직

데이터 관리

거버넌스

인터넷

레이어 5 — 퍼블릭 블록 체인

이더리움 EVM

↓ Tx Broadcast / 이벤트 감지

레이어 4 — 인가 VASP

WalletOne

S5 핵심 구간: 레이어 2 게이트웨이의 **Webhook Receiver**(수신 → 202 즉시 응답) → **Redis Event Pipeline**(비동기 처리).

코드 구조 ↔ 레이어 매핑

CODE PACKAGES vs ARCHITECTURE LAYERS — 1:1 매핑이 아닌 이유

패키지 / 앱 → 레이어

코드	레이어
issuer-service	L3 코어 서비스 (런타임 진입점)
event-engine	L2 게이트웨이 (이벤트 파이프라인)
vasp	L2 (IVASAdapter) + L3 (OutboxWorker)
chain-adapters	L2 게이트웨이 (IBlockchainAdapter)
compliance	L3 코어 서비스
core-banking	L3 코어 서비스
shared	전 레이어 공통
blockchain/	L5 퍼블릭 블록체인 (Solidity)
internal-ledger	L3 코어 서비스 (Java, 영구 원장 · 금융 감사 로그)

왜 1:1이 아닌가?

레이어 = 런타임 신뢰 경계

누가 누구를 호출할 수 있는지를 네트워크/방화벽으로 강제. 어디에 배포되는가의 문제.

패키지 = 코드 응집 단위

코드를 어떻게 나누고 재사용할지의 문제. 한 패키지가 여러 레이어에 걸칠 수 있음.

현재 상태: 모든 패키지가 issuer-service 하나로 번들되어 배포됨. 레이어 경계는 실제 네트워크 격리가 아닌 **설계 의도**. 레이어를 실제로 강제하려면 각 레이어가 별도 프로세스/컨테이너로 분리되어야 함.

Phase 1 — Webhook 발신자 구분

WHO SENDS WEBHOOKS TO WebhookReceiver IN PHASE 1

[Phase 1 — 현재 구현] 이 모듈은 VASP(월렛원) 위탁 아키텍처를 기반으로 합니다.

ACTIVITY_ACHIEVED

- 발신자: 내부망 (교보 앱 서버)
- 사용자 활동 달성 알림
- WebhookReceiver가 수신
- → Queue 적재 → NFT 발행 워크플로우

NFT_ISSUED

- 발신자: 월렛원 (외부 VASP)
- 온체인 TX 확정 후 발행 완료 콜백
- WebhookReceiver가 수신
- → 원장 업데이트 트리거

Phase 3 예고: 직접 Custody 운영 시 NFT_ISSUED는 VASP Webhook이 아니라 ChainEventListener가 블록체인 이벤트를 직접 구독해 처리한다.

이 세션이 답하는 질문

THREE KEY QUESTIONS — WHY 202 AND QUEUE MATTER

Q1. Webhook을 받자마자 DB에 저장하면 안 되는가?

DB 장애 순간 Webhook이 오면 이벤트가 영구적으로 사라진다.

→ 발신자가 응답을 기다리다 **timeout** 후 재전송해도, 우리 쪽이 복구될 때까지 계속 유실된다.

Q2. 202를 즉시 반환하면 처리가 안 된 것 아닌가?

"나 받았어 (Accepted)"와 "나 처리했어 (Done)"는 다르다.

→ 202는 "받았으니 내가 알아서 처리할게"라는 의미다. 처리는 **Queue**를 통해 비동기로, 실패해도 재처리 가능하다.

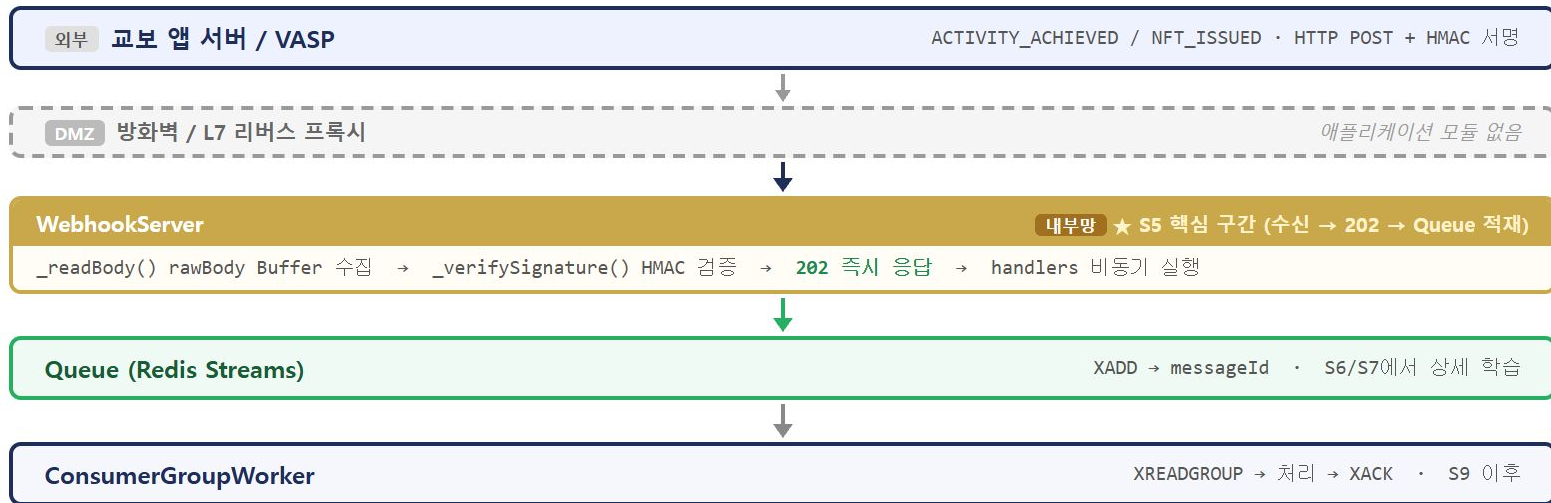
Q3. Queue가 없으면 어떤 일이 생기는가?

처리 실패 시 재시도 메커니즘이 없다. Consumer가 느려지면 수신 속도가 함께 느려진다.

→ 수신과 처리를 분리해야 각자 독립적으로 확장할 수 있다.

전체 흐름에서의 위치

이벤트 수신 파이프라인 — FULL ARCHITECTURE OVERVIEW



이 세션의 구간

SESSION SCOPE — FROM APP SERVER TO QUEUE ENTRY



중요: 블록체인 이벤트 (NFT 발행 확인)는 WebhookServer가 아닌 ChainEventListener가 수신한다.

Core Banking은 내부망이 아웃바운드로 알림을 보내는 대상 — WebhookServer를 호출하지 않는다.

1부 — 즉시 처리 패턴의 위험

1-1. WRONG PATTERN — SYNCHRONOUS PROCESSING IN WEBHOOK HANDLER

많은 개발자가 처음 Webhook을 구현할 때 이렇게 짤다 — 이 코드는 금융 시스템에서 절대 사용해선 안 된다.

```
// ❌ 잘못된 구현 - 즉시 처리 패턴
app.post('/webhook', async (req, res) => {
  const payload = req.body;

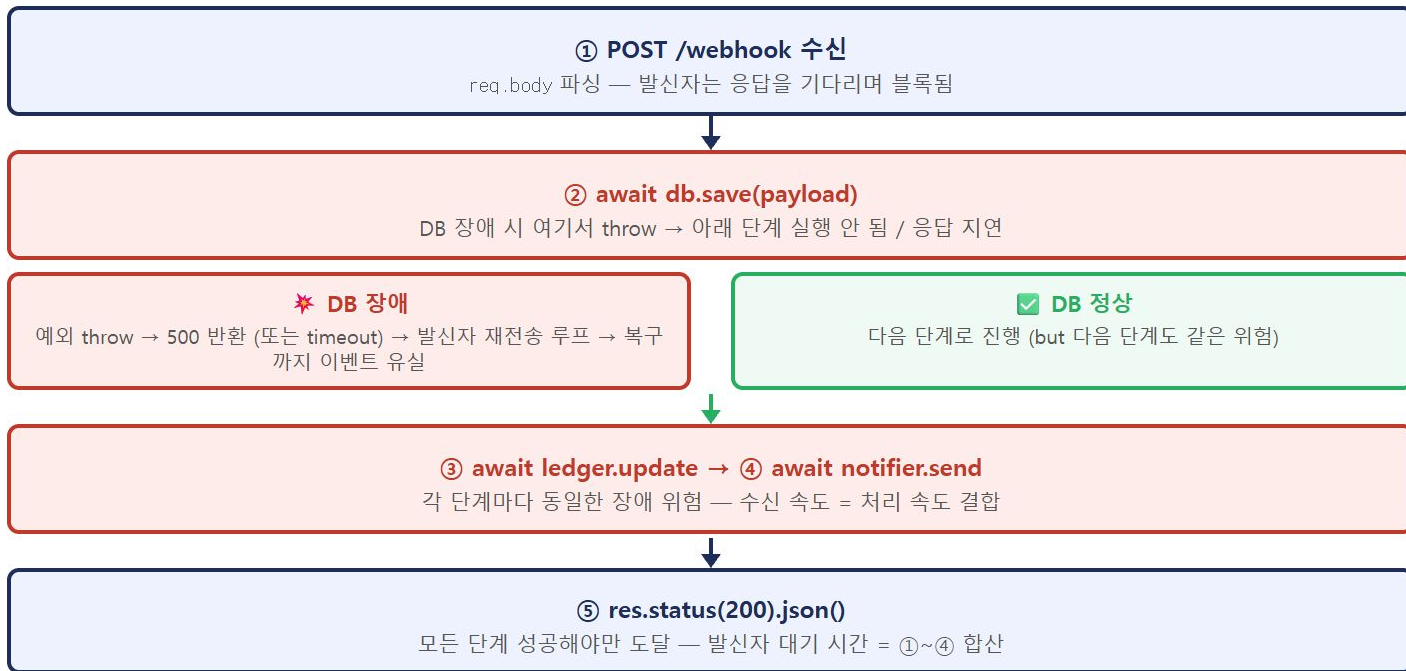
  await db.save(payload);           // ← 여기서 장애 나면?
  await ledger.update(payload);      // ← 여기서 timeout 나면?
  await notifier.send(payload);     // ← 여기서 실패하면?

  res.status(200).json({ ok: true });
});
```

문제	설명	결과
이벤트 유실	DB 장애 시 이벤트 저장 불가	회계 불일치, NFT 누락
느린 응답	처리 완료까지 응답 지연	발신자 timeout → 재전송
결합도	수신 속도 = 처리 속도	처리가 느리면 수신도 막힘

1부 — 즉시 처리 패턴의 위험 — 실행 흐름

SYNCHRONOUS WEBHOOK: REQUEST ENTERS, EVERYTHING BLOCKS, ONE FAILURE KILLS ALL



핵심: 동기 처리 패턴은 수신·처리·응답이 하나의 체인 — 어느 한 단계 장애가 이벤트 영구 유실로 직결된다.

즉시 처리 — 장애 타임라인

FAILURE TIMELINE — DB OUTAGE DURING SYNCHRONOUS WEBHOOK PROCESSING

시각	발신자(교보 앱 서버)	수신자(우리 서버)
T+0	POST /webhook 전송	요청 수신
T+1		db.save() 호출 중...
T+2		[DB 연결 오류 발생] ← 🌟
T+3	(응답 없음 - 대기 중)	
T+30	timeout → 재전송 시도	서버가 아직 장애 중
T+60	재전송 포기	이벤트 영구 유실 ← 💀

금융 시스템에서 이 패턴이 치명적인 이유: DB 장애 시 이벤트가 영구 유실된다. 발신자가 재전송을 포기하면 NFT 미발행·원장 불일치가 자동 생성된다. 이는 자동 복구 불가능한 상태다.

핵심 통찰: 발신자의 재전송 간격/횟수 정책은 우리가 제어할 수 없다. 우리 시스템의 복구 시간이 발신자의 재전송 한도를 초과하면 이벤트는 영구 유실된다.

1-2. 올바른 방식 — 202 + Queue 패턴

CORRECT PATTERN — ACCEPT IMMEDIATELY, PROCESS ASYNCHRONOUSLY

```
// ✅ 올바른 구현 - 202 + Queue 패턴
app.post('/webhook', async (req, res) => {
  const rawBody = await readRawBody(req); // Buffer로 수집

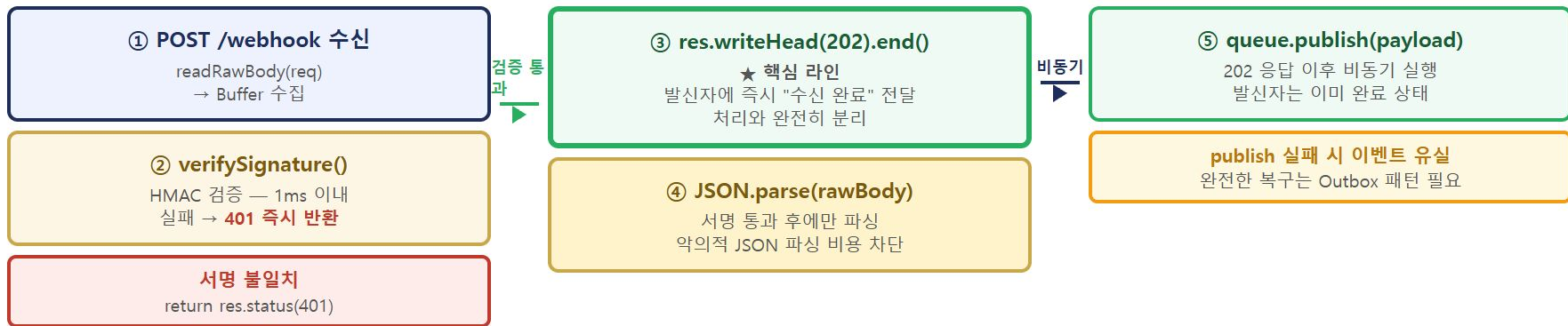
  // 서명 검증 (빠름 - 1ms 이내)
  if (!verifySignature(rawBody, req.headers['x-kyobo-signature'])) {
    return res.status(401).end();
  }

  // 즉시 202 응답 ← 발신자에게 "받았어" 확인
  res.status(202).end();

  // Queue 적재는 비동기 ← 응답 이후 처리
  const payload = JSON.parse(rawBody.toString('utf8'));
  await queue.publish(payload); // 실패해도 발신자는 이미 202 받음
});
```

1-2. 올바른 방식 — 202 + Queue 패턴 — 실행 흐름

RECEIVE → VERIFY → 202 IMMEDIATELY → QUEUE ASYNC: DECOUPLED RESPONSIBILITY



핵심: 202 응답과 Queue 적재는 시간 순서가 분리된다 — 수신 책임과 처리 책임이 독립적으로 각자 실패·복구 가능하다.

1-2. Outbox 패턴 — publish 실패 구멍 막기

WHY DB + QUEUE TOGETHER: DB AS RECOVERY EVIDENCE, QUEUE AS FAST TRIGGER

❌ Queue만 발행하는 경우

③ 202 응답
발신자 "전송 완료" 인식

⑤ queue.publish() 실패
payload는 메모리(req.body)에만 존재

✱ 함수 종료 → 메모리 해제
재시도 근거 없음 → 이벤트 영구 유실

Outbox



✅ Outbox 패턴

DB 트랜잭션 안에서
INSERT INTO outbox (payload, status='PENDING')
→ 202 응답 — Queue는 아직 안 함

Outbox Poller (별도 프로세스)
SELECT WHERE status='PENDING'
→ queue.publish() → status='DONE'

publish 실패해도
DB에 PENDING 잔류 → Poller가 자동 재시도

DB (Outbox 테이블)

이벤트가 존재했다는 증거
Queue 실패 시 복구 근거

Queue (Redis Streams 등)

Consumer에게 빠르게 알리기
DB 폴링 없이 실시간 처리 트리거

핵심: DB 저장은 Queue를 대체하지 않는다 — Queue가 실패했을 때 재발행할 근거를 남기는 것이다. 둘의 역할은 분리되어 있다.

202 패턴 — 타임라인 비교

TIMELINE COMPARISON — SYNCHRONOUS VS ASYNC 202 PATTERN

✅ 202 패턴 타임라인:

시각 발신자(교보 앱 서버)

수신자(우리 서버)

T+0 POST /webhook 전송

요청 수신

T+1

서명 검증 (1ms 이내)

T+2

202 응답 ← 즉시 반환 ✅

T+3 202 수신 - "OK, 전달 완료"

[비동기] Queue.publish()

→ Redis XADD → messageId 기록

이후 Consumer가 처리

❌ 즉시 처리 결과:

DB 장애 → 응답 지연 → 발신자 timeout → 재전송 루프 → 이벤트 유실

✅ 202 패턴 결과:

DB 장애와 무관하게 202 즉시 반환 → 발신자 안심 → Queue가 내구성 담당

2부 — WebhookServer 클래스 구조 전체 읽기

2-1. CLASS STRUCTURE — TYPES, CONSTRUCTOR, PUBLIC API

```
export type WebhookPayload = {
  eventType: string;
  data: Record<string, unknown>;
  timestamp: number;
  requestId: string; // 멍등성 키 - 중복 처리 방지에 사용
};

export type WebhookHandler = (payload: WebhookPayload) => Promise<void>;

export class WebhookServer {
  private server: http.Server;
  private handlers: Map<string, WebhookHandler[]> = new Map();

  constructor(private readonly config: {
    port: number;
    secret: string; // HMAC 시크릿 - 환경 변수로 주입
    maxBodyKb: number; // payload 크기 제한
  }) {
    this.server = http.createServer(this._handleRequest.bind(this));
  }

  // 핸들러 등록 - 체이닝 가능
  on(eventType: string, handler: WebhookHandler): this { ... }
  listen(): Promise<void> { ... }
  close(): Promise<void> { ... }

  private async _handleRequest(req, res): Promise<void> { ... } // ← 이 세션
  private _readBody(req): Promise<Buffer> { ... } // ← 이 세션
  private _verifySignature(rawBody, sig): boolean { ... } // ← S6 상세
}
```


2부 — WebhookServer 클래스 구조 전체 읽기 — 실행 흐름

CLASS ANATOMY: TYPES → CONSTRUCTOR → PUBLIC API → PRIVATE METHODS

타입 계층

① WebhookPayload

eventType · data · timestamp

requestId = 멍등성 키

중복 처리 방지에 사용

② WebhookHandler

(payload) → Promise<void>

비동기 함수 시그니처

핸들러 타입 계약

클래스 상태

③ private server

http.Server 인스턴스

constructor에서 생성

_handleRequest 바인딩

④ private handlers

Map<eventType, Handler[]>

같은 이벤트에 N개 핸들러

체이닝으로 등록

메서드 계층

⑤ Public: on / listen / close

on() → this 반환 (체이닝)

listen() → 포트 바인딩

⑥ Private: _handleRequest

★ 이 세션 핵심

요청 전체 처리 파이프라인

⑦ Private: _readBody / _verifySignature

Buffer 수집 · HMAC 검증

핵심: 타입 → 상태 → Public API → Private 구현 순으로 읽으면 클래스 전체 책임이 한눈에 보인다. private 메서드 3개가 이 세션의 전부다.

_handleRequest — 6단계 흐름

2-2. THE HEART OF WebhookServer — STEP BY STEP

단계	코드	설계 이유
Step 1	<code>if (req.method !== 'POST') → 405</code>	POST 메서드만 허용. 다른 메서드는 즉시 거부
Step 2	<code>rawBody = await _readBody(req)</code>	<code>rawBody</code> 를 Buffer로 수집. string 아닌 Buffer인 이유 → 슬라이드 19
Step 3	<code>if (!_verifySignature(rawBody, sig)) → 401</code>	HMAC 서명 검증. 신뢰의 관문 — 통과 전까지 JSON 파싱 안 함
Step 4	<code>payload = JSON.parse(rawBody.toString('utf8'))</code>	서명 통과 후 파싱. 악의적 JSON 파싱 비용 차단
Step 5 ★	<code>res.writeHead(202).end()</code>	즉시 응답 — 이 줄이 핵심. 발신자 timeout 방지
Step 6	<code>await Promise.allSettled(handlers...)</code>	202 이후 비동기 실행. 하나 실패해도 나머지 계속

_handleRequest — 완성 코드

COMPLETE IMPLEMENTATION — ANNOTATED WITH DESIGN RATIONALE

```
private async _handleRequest(
  req: http.IncomingMessage,
  res: http.ServerResponse,
): Promise<void> {
  if (req.method !== 'POST') { res.writeHead(405).end(); return; } // Step 1

  try {
    const rawBody = await this._readBody(req); // Step 2
    const signature = req.headers['x-kyobo-signature'] as string ?? '';

    if (!this._verifySignature(rawBody, signature)) { // Step 3
      res.writeHead(401).end('invalid signature');
      return;
    }

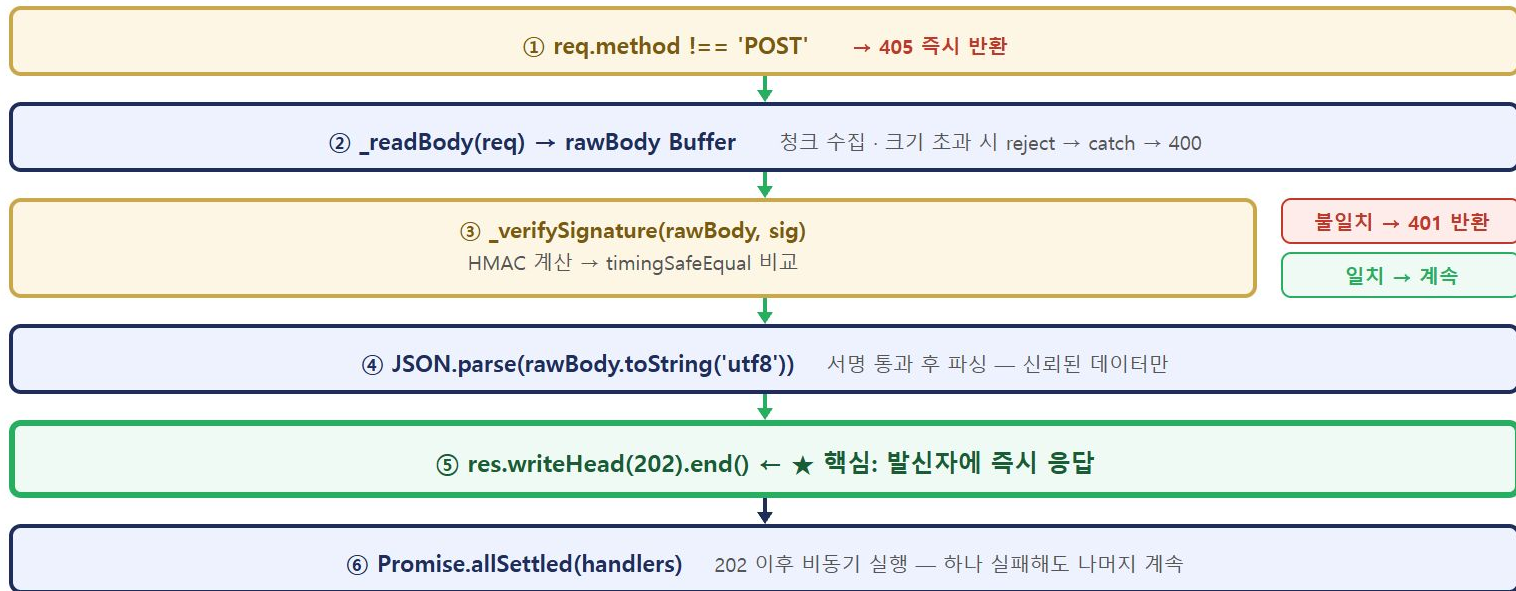
    const payload: WebhookPayload = JSON.parse(rawBody.toString('utf8')); // Step 4

    res.writeHead(202).end(); // Step 5 ← 핵심

    const handlers = this.handlers.get(payload.eventType) ?? [];
    await Promise.allSettled(handlers.map(h => h(payload))); // Step 6
  } catch (err) {
    if (!res.headersSent) res.writeHead(400).end();
    console.log('Webhook server error:', err);
  }
}
```

_handleRequest — 완성 코드 — 실행 흐름

6-STEP PIPELINE: METHOD CHECK → BODY → HMAC → PARSE → 202 → HANDLERS



핵심: ①~③은 방어 게이트(빠른 거부), ④는 신뢰 후 파싱, ⑤가 응답 책임 완료, ⑥은 처리 책임 시작 — 두 책임이 한 함수 안에서 명확히 분리된다.

Step 5와 6의 순서 — 왜 중요한가

ORDER MATTERS — 202 BEFORE HANDLER EXECUTION IS NON-NEGOTIABLE

❌ 잘못된 순서

```
// handlers 완료 후 202
await Promise.allSettled(handlers...)
res.writeHead(202).end();
```

- 처리 시간(수백ms~수초)만큼 발신자 대기
- DB 장애 시 202 응답 자체가 안 됨
- 이벤트 재전송 루프 발생

✅ 올바른 순서

```
// 즉시 202 후 handlers
res.writeHead(202).end()
await Promise.allSettled(handlers...)
```

- 발신자는 1~5ms 내에 응답 받음
- 처리 실패는 Queue 레이어가 담당
- 수신 성공과 처리 성공은 분리된 책임

Node.js 동작 원리: `res.writeHead(202).end()` 이후에도 함수 실행은 계속된다. 응답이 이미 전송된 후 `async handler`가 실행된다. 이 비동기 특성을 의도적으로 활용하는 패턴이다.

2-4. _readBody — rawBody를 Buffer로 유지하는 이유

WHY BUFFER NOT STRING — HMAC REQUIRES ORIGINAL BYTES

```
private _readBody(req: http.IncomingMessage): Promise<Buffer> {
  return new Promise((resolve, reject) => {
    const chunks: Buffer[] = [];
    let size = 0;

    req.on('data', (chunk: Buffer) => {
      size += chunk.length;
      if (size > this.config.maxBodyKb * 1024) {
        reject(new Error('payload too large'));
        return;
      }
      chunks.push(chunk);
    });

    req.on('end', () => resolve(Buffer.concat(chunks)));
    req.on('error', reject);
  });
}
```

❌ string으로 받으면:

req.setEncoding('utf8') 후 자동 디코딩 → Buffer.from(body) 재변환 필요 → 인코딩 오류 가능성 (멀티바이트)

✅ Buffer로 받으면:

원본 바이트 보존 → HMAC 계산 시 발신자 서명과 일치 → JSON 파싱은 toString('utf8')로 명시적 변환

2-4. _readBody — rawBody를 Buffer로 유지하는 이유 — 실행 흐름

STREAM → CHUNK ACCUMULATION → SIZE GUARD → BUFFER.CONCAT → RESOLVE



핵심: Buffer.concat()으로 원본 바이트를 보존해야 발신자가 서명한 바이트와 동일한 데이터로 HMAC을 재계산할 수 있다 — string 변환 시 인코딩 오류로 서명 불일치 발생 가능.

2-3. Promise.allSettled vs Promise.all

INDEPENDENT HANDLER EXECUTION — WHY allSettled IS THE RIGHT CHOICE

❌ Promise.all — 하나 실패 시 에러 throw

```
await Promise.all(handlers.map(h => h(payload)));  
// handler1: Queue 적재 ✅  
// handler2: AuditLog ← 실패  
// → handler1은 이미 실행됐지만 에러 throw  
// → 오해 유발, 나머지 핸들러 상태 불명확
```

✅ Promise.allSettled — 모두 실행, 결과 개별 확인

```
await Promise.allSettled(handlers.map(h => h(payload)));  
// handler1: { status: 'fulfilled', value: undefined }  
// handler2: { status: 'rejected', reason: Error }  
// → 각 핸들러 독립 실행  
// → 실패 로그만 기록, 나머지 계속
```

설계 원칙: 각 핸들러는 독립적인 책임을 가진다. Queue 적재 핸들러와 AuditLog 핸들러는 서로 의존하지 않는다. 하나의 실패가 다른 핸들러를 막아서는 안 된다.

handlers.get(eventType) ?? [] — 미등록 eventType이 오면 빈 배열 반환. Promise.allSettled([])는 아무것도 실행하지 않는다. 202 응답은 정상 반환된다.

핸들러 등록 — 체이닝 패턴

HANDLER REGISTRATION — SAME EVENT TYPE CAN HAVE MULTIPLE HANDLERS

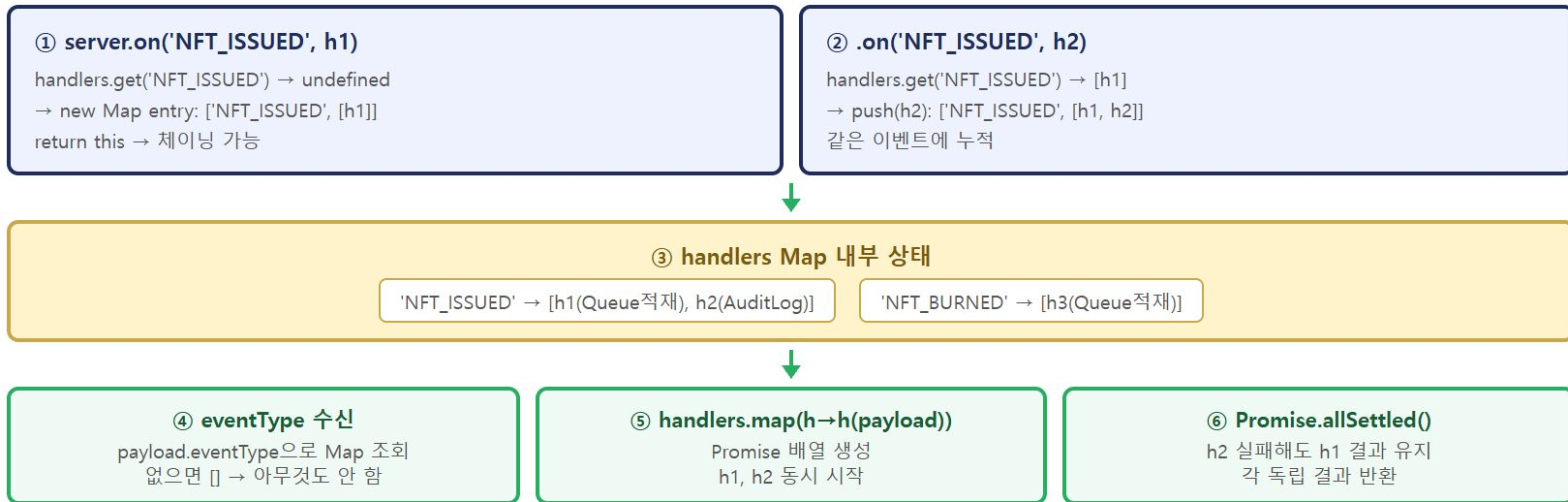
```
const server = new WebhookServer({ port: 3000, secret: process.env.WEBHOOK_SECRET!, maxBodyKb: 64 });

// 같은 eventType에 여러 핸들러 등록 가능 (체이닝)
server
  .on('NFT_ISSUED', async (payload) => {
    // 핸들러 1: Queue에 적재
    await publisher.publish({
      streamKey: 'kyobo:events', eventType: payload.eventType,
      payload: payload.data, requestId: payload.requestId,
      txHash: payload.data['txHash'] as string,
      blockNumber: payload.data['blockNumber'] as number,
    });
  })
  .on('NFT_ISSUED', async (payload) => {
    // 핸들러 2: AuditLog 기록 (독립 실행)
    await auditLog.record(payload);
  })
  .on('NFT_BURNED', async (payload) => {
    await publisher.publish({ /* NFT_ISSUED와 동일 구조 */ });
  });

await server.listen();
```


핸들러 등록 — 체이닝 패턴 — 실행 흐름

HANDLER REGISTRATION → MAP STORAGE → EVENT DISPATCH → ALLSETTLED EXECUTION



핵심: `on()`은 Map에 핸들러를 누적 등록하고 `this`를 반환해 체이닝을 가능하게 한다. 이벤트 수신 시 해당 `eventType`의 모든 핸들러가 독립 실행된다.

3부 — 202 패턴의 설계 철학

3-1. HTTP RESPONSE CODE SEMANTICS — WHAT EACH CODE MEANS

코드	의미	사용 시점	예시
200 OK	처리 완료	동기 처리가 완료된 경우	GET 요청, 동기 API
201 Created	리소스 생성 완료	POST로 새 자원 생성 시	REST API 리소스 생성
202 Accepted ★	수신 확인, 처리는 나중에	비동기 처리 예약 시	Webhook 수신, 배치 작업
400 Bad Request	요청 형식 오류	JSON 파싱 실패 등	malformed body
401 Unauthorized	인증 실패	서명 검증 실패	HMAC 불일치
405 Method Not Allowed	메서드 미허용	POST 외 메서드 거부	GET, PUT 등 거부

202의 의미: "나 받았어 (Accepted)" ≠ "나 처리했어 (Done)" — 이 두 개념의 분리가 202 패턴의 핵심이다.

3-2. 202 패턴이 만드는 시스템 특성

SYSTEM PROPERTIES CREATED BY 202 + QUEUE PATTERN

202 즉시 응답

발신자 측

"전달 성공" 확인
재전송 루프 없음
timeout 없음

수신자 측

처리 실패는 내부 재시도
수신 속도와 처리 속도 분리
독립 확장 가능

시스템 전체

수신 처리량 $\neq$ 처리 처리량
각자 독립적으로 확장
장애 격리

Queue 없이 202만 쓰면?

비동기 처리 중 실패 → 재시도 불가 → 이벤트 유실. 202는 반드시 Queue와 함께 사용해야 의미가 있다.

실습 — 전체 목표와 구성

PRACTICE SESSION — 30 MINUTES · 4 STEPS

실습 목표: WebhookServer의 `_handleRequest` 구조를 직접 따라가며, Queue 적재 실패 시 어떻게 처리할지 정책을 결정한다.

단계	시간	목표	파일
Step 1	5분	코드 읽기 — <code>_handleRequest</code> 이해	WebhookServer.ts
Step 2	10분	핸들러 등록 → 서버 기동 스켈레톤 완성	exercises/S05_server.ts
Step 3	10분	HMAC 서명 생성 → Webhook 전송 → 응답 확인	S05_make_sig.ts
Step 4	5분	Queue 적재 실패 정책 결정 (토론)	—

완료 기준: WebhookServer 기동 → 올바른 서명 202 / 잘못된 서명 401 / 서명 없음 401 / POST 외 405 — 4가지 확인

실습 Step 1 — 코드 읽기 (5분)

READ CODE · ANSWER THREE QUESTIONS BEFORE PROCEEDING

```
# 실제 파일 열기 (PowerShell)
```

```
code dmz/packages/event-engine/src/webhook/WebhookServer.ts
```

아래 질문에 답하면서 읽어라. 코드를 이해하는 가장 빠른 방법은 "이 코드가 없으면 어떻게 되는가"를 자문하는 것이다.

읽기 질문 (체크하면서 진행)

- `_handleRequest`의 `try/catch`가 잡는 에러는 어떤 종류인가?
- 202를 보낸 후 `handlers` 실행이 실패하면 `catch`에 걸리는가?
- `maxBodyKb` 초과 시 어떤 응답이 가는가?

정답 힌트

- `try/catch`: `_readBody`의 `size` 초과, `_verifySignature`의 `Buffer` 오류, `JSON.parse` 실패
- `handlers` 실패: `res.headersSent=true`이므로 400 응답 안 나감. 에러 로그만
- `maxBodyKb` 초과: `_readBody`에서 `reject` → `catch`에서 400 반환

실습 Step 2 — 핸들러 등록 · 서버 기동 (10분)

SKELETON → IMPLEMENT · exercises/S05_server.ts

```
import { WebhookServer } from './webhook/WebhookServer';

const server = new WebhookServer({
  port: 3001,
  secret: 'dev-secret-kyobo',
  maxBodyKb: 64,
});

// TODO 1: 'NFT_ISSUED' 이벤트에 핸들러 등록
// 핸들러 내용: console.log로 payload 출력 + 200ms 슬립
server.on('NFT_ISSUED', async (payload) => {
  // TODO: 구현
});

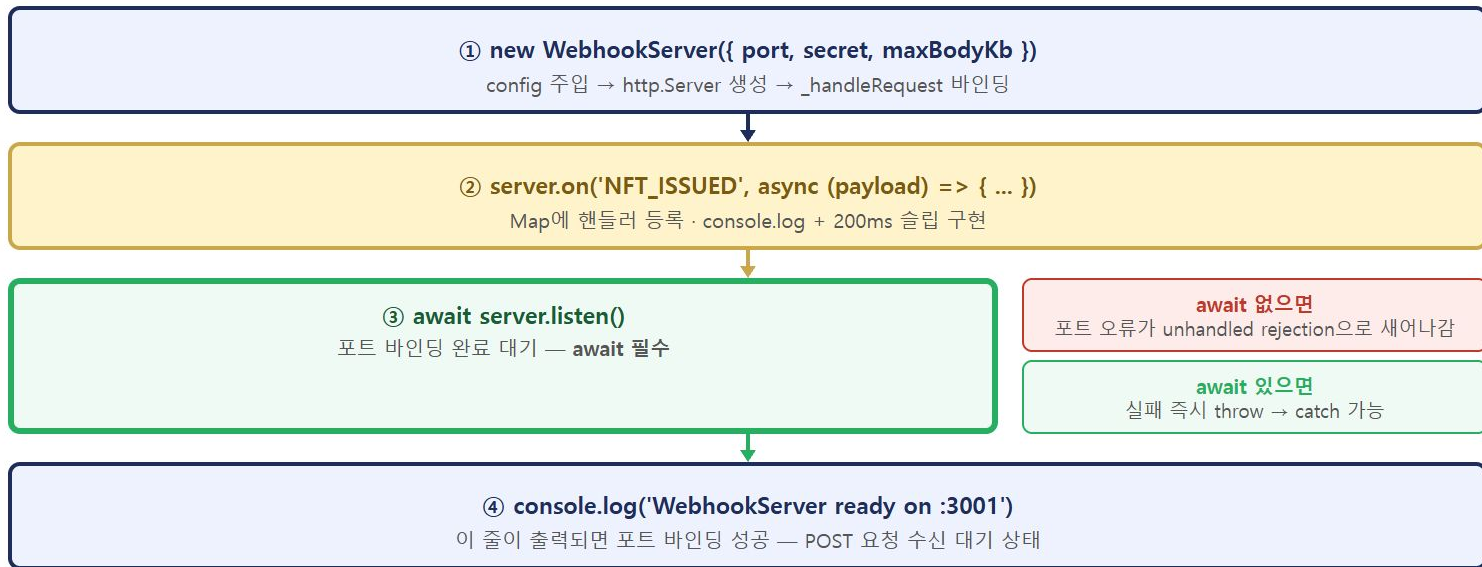
// TODO 2: 서버 listen (await 필수 - 포트 오류를 즉시 잡기 위해)
// TODO: 구현

console.log('WebhookServer ready');
```

핵심 포인트: await server.listen() — await 없으면 포트 충돌이나 권한 에러가 unhandled rejection으로 새어나간다.

실습 Step 2 — 핸들러 등록 · 서버 기동 — 실행 흐름

SKELETON LOGIC: INSTANTIATE → REGISTER HANDLER → AWAIT LISTEN → READY



핵심: `IIFE (async () => { await server.listen(); })()` 패턴 — 최상위 `await`가 불가능한 환경에서 `async` 문맥을 즉시 만들어 `listen` 결과를 동기적으로 처리한다.

실습 Step 2 — 답안 & 문법 포인트

ANSWER — KEY SYNTAX EXPLAINED

```
import { WebhookServer } from './webhook/WebhookServer';

const server = new WebhookServer({ port: 3001, secret: 'dev-secret-kyobo', maxBodyKb: 64 });

server.on('NFT_ISSUED', async (payload) => {
  console.log('[handler] NFT_ISSUED received:', JSON.stringify(payload, null, 2));
  await new Promise(r => setTimeout(r, 200)); // JS/TS sleep 관용구
  console.log('[handler] NFT_ISSUED processed');
});

(async () => {
  await server.listen();
  console.log('WebhookServer ready on :3001');
})();
```

항목	스켈레톤	답안
payload 출력	console.log(payload)	JSON.stringify(null, 2) — 깊은 객체도 보임
슬립 표현	막연히 200ms	new Promise(r => setTimeout(r, 200)) 관용구
listen	server.listen()	await server.listen() — 실패 즉시 throw

실습 Step 3 — HMAC 서명 생성 (10분)

S05_make_sig.ts — GENERATE SIGNATURE FOR CURL/PowerShell TESTING

```
// 실행: npx tsx src/exercises/S05_make_sig.ts
import crypto from 'crypto';

const payload = JSON.stringify({
  eventType: 'NFT_ISSUED',
  data:      { tokenId: '42', owner: '0xABCD' },
  timestamp: Date.now(),
  requestId: 'test-001',
});
const sig = crypto.createHmac('sha256', 'dev-secret-kyobo')
  .update(Buffer.from(payload)) // ← Buffer.from() 필수 (인코딩 안전)
  .digest('hex');
console.log('signature:', sig);
console.log('payload: ', payload);
```

PowerShell로 WebhookServer 호출 - S05_make_sig.ts 출력 복붙

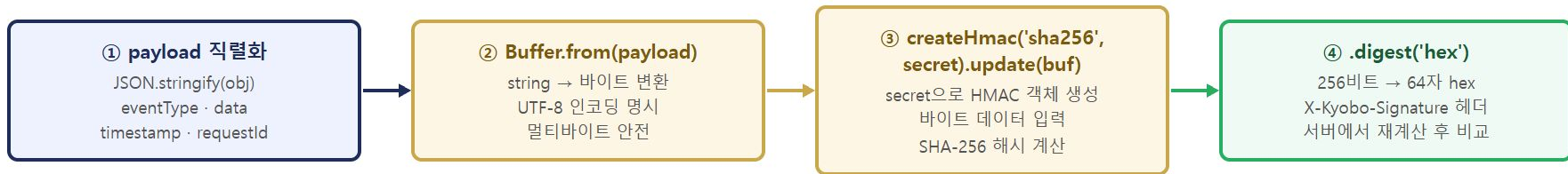
```
npx tsx src/exercises/S05_make_sig.ts
```

출력된 명령어 복붙해서 실행:

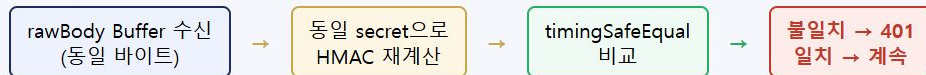
```
Invoke-WebRequest -Uri http://localhost:3001 -Method POST `
-headers @{ "Content-Type" = "application/json"; "X-Kyobo-Signature" = "a3f2..." } `
-Body '{"eventType":"NFT_ISSUED",...}' | Select-Object -ExpandProperty StatusCode
```

실습 Step 3 — HMAC 서명 생성 — 실행 흐름

PAYLOAD SERIALIZE → HMAC COMPUTE → HEX OUTPUT → HTTP HEADER ATTACH



서버 측 검증 (verifySignature) 대응 흐름



핵심: 클라이언트와 서버가 동일한 secret + 동일한 바이트로 HMAC을 계산해야 서명이 일치한다 — Buffer.from()이 두 곳 모두에서 필수인 이유다.

HMAC 서명 생성 코드 — 문법 라인별 해설

HMAC SIGNATURE GENERATION — SYNTAX BREAKDOWN

`crypto.createHmac('sha256', secret)`

- HMAC 생성기 객체 만들
- 첫 번째 인자: 해시 알고리즘 (sha256/sha512)
- 두 번째 인자: secret key
- secret 없으면 서명 위조 불가 = 인증 효과

`.digest('hex')`

- 해시 계산 확정 + 결과 출력
- 'hex' = 16진수 문자열
- SHA-256 결과 = 256비트 = 64자 hex 문자열
- 다른 옵션: 'base64', 'binary', Buffer

`.update(Buffer.from(payload))`

- 메서드 체이닝 패턴
- Buffer.from() = 문자열 → 바이트 변환
- 명시적 Buffer → 인코딩 안전 (UTF-8 보장)
- HMAC 내부는 바이트 단위로 동작

`Date.now() + requestId` 역할

- timestamp: 오래된 요청 차단 (재전송 공격 방어)
- requestId: 같은 요청 중복 처리 방지 (idempotency)
- 운영 환경: secret은 반드시 env/Vault에서 주입
- 검증 시 timingSafeEqual 필수 (=== 타이밍 공격 취약)

실습 Step 3 — 예상 응답 결과

EXPECTED RESULTS — FOUR SCENARIOS TO VERIFY

테스트 시나리오 4가지

시나리오	예상 응답
올바른 서명	202 Accepted
잘못된 서명 (0000...)	401 Unauthorized
서명 헤더 없음	401 Unauthorized
POST 외 메서드 (GET)	405 Method Not Allowed

확인 방법 (PowerShell)

```
# 올바른 서명: 202
Invoke-WebRequest -Uri http://localhost:3001 `
  -Method POST `
  -Headers @{
    "Content-Type" = "application/json"
    "X-Kyobo-Signature" = $sig
  } -Body $payload | % StatusCode

# 서명 없음: 401
Invoke-WebRequest -Uri http://localhost:3001 `
  -Method POST `
  -Headers @{"Content-Type"="application/json"} `
  -Body $payload | % StatusCode
```

4가지 결과를 모두 확인하면 WebhookServer의 인증 + 라우팅 로직을 완전히 이해한 것이다.

실습 Step 4 — Queue 적재 실패 정책 (5분)

DESIGN DECISION — WHAT TO DO WHEN REDIS IS DOWN AFTER 202

시나리오: 발신자에게 202를 이미 보냈다. 이후 handler() → publisher.publish() 호출 → Redis 연결 오류 → publish() throw. Promise.allSettled에서 status: 'rejected' 확인됨.

정책 A: 로그만 기록

- 구현 단순
- 이벤트 유실 발생
- 금융 시스템에서 절대 불가

정책 B: 인메모리 재시도 큐

- 서버 재시작 시 유실
- 완화책이지 해결책 아님
- 단기 장애에만 효과적

정책 C: 로컬 파일/DB Fallback

- Redis 복구 시 재처리
- 구현 복잡도 증가
- 단일 장애점 감소

정책 D: Redis HA 구성 ★

- Redis Sentinel / Cluster
- 단일 장애점 근본 제거
- 교보 프로젝트 선택: D + C 조합

핵심 정리 & 다음 세션 예고

KEY CONCEPTS · NEXT SESSION PREVIEW — Redis Streams

개념	핵심	기억할 것
202 Accepted	"받았어, 처리는 내가 알아서"	발신자 timeout 방지
rawBody Buffer	JSON 재직렬화 없이 원본 바이트 보존	서명 불일치 방지
Promise.allSettled	독립 핸들러 모두 실행	하나 실패해도 나머지 실행
Queue 분리 이유	이벤트 내구성 / 독립 확장 / 재처리	202만으로는 불충분
즉시 처리 금지	DB 장애 시 이벤트 영구 유실	금융 시스템 절대 원칙

S6 예고 — Redis Streams 내부 구조:

Pub/Sub vs Queue vs Streams — 왜 Streams인가? · At-least-once 보장이란?

XREADGROUP으로 메시지를 읽으면 PEL에 등록된다. 처리 완료 후 XACK를 보내야 PEL에서 제거된다. Consumer가 crash해도 PEL에 남아있던 메시지는 재수신된다. 이 메커니즘의 내부 구조를 S6에서 완전히 해부한다.